

MEA(R)N Stack Web Development (303105385)

Unit 1 : Introduction to Web Development and the MEAN Stack

Prof. Sumersing Patil

Assistant Professor

Computer Science & Engineering



Outline

- Overview of web development
- Introduction to the MEAN stack
- Setting up the development environment



Overview of web development

- Web development refers to the process of creating, building, and maintaining websites or web applications.
- It involves several disciplines, tools, and technologies to ensure the development of functional, visually appealing, and user-friendly digital products.
- Web development can be categorized into front-end development, back-end development, and full-stack development.



Overview of web development

1. Types of Web Development

a) Front-End Development

- Focus: User interface (UI) and user experience (UX).
- Technologies:
 - HTML (HyperText Markup Language): Structures content.
 - CSS (Cascading Style Sheets): Styles and layouts content.
 - JavaScript: Adds interactivity and dynamic features.
 - Frameworks/Libraries: React, Angular, Vue.js.



Overview of web development

- Tools: Browser Developer Tools, code editors (VS Code, Sublime Text).
- Responsibilities:
 - Designing responsive, mobile-friendly layouts.
 - Ensuring cross-browser compatibility.
 - Enhancing website accessibility.



Overview of web development

b) Back-End Development

- Focus: Server-side logic and database management.
- Technologies:
 - Programming Languages: Python, PHP, Ruby, Java, Node.js.
 - Databases: MySQL, MongoDB, PostgreSQL.
 - Frameworks: Django, Flask, Laravel, Express.js.
- Tools: Server management (Apache, Nginx), RESTful APIs, cloud platforms (AWS, Azure).



Overview of web development

- Responsibilities:
 - Handling server-side operations.
 - Building and managing APIs.
 - Managing databases and ensuring secure data flow.
- c) Full-Stack Development
 - Focus: Both front-end and back-end development.
 - Technologies: Combines front-end and back-end technologies.
 - Responsibilities:
 - Overseeing the entire web development process.
 - Bridging the gap between UI/UX and server-side logic.

Overview of web development

2. Key Components of Web Development

a) Web Design

- Involves creating wireframes, prototypes, and the visual layout of websites.
- Tools: Figma, Adobe XD, Sketch.

b) Web Hosting

- Web hosting services provide the infrastructure to store and serve web pages.
- Types: Shared hosting, VPS hosting, cloud hosting.



Overview of web development

c) Domain Registration

- Acquiring a unique domain name to make the website accessible (e.g., www.example.com).

d) Version Control

- Tools like Git and platforms like GitHub or GitLab help track and manage code changes.



Overview of web development

e) Security

- Ensuring secure user data through SSL certificates, encryption, and secure coding practices.
- Techniques: Firewalls, regular updates, secure authentication methods.

f) Search Engine Optimization (SEO)

- Optimizing websites to rank higher on search engine results pages (SERPs).
- Techniques: Meta tags, keyword optimization, and mobile-friendly designs.



Overview of web development

3. Stages of Web Development

- 1) Planning: Define goals, target audience, and website requirements.
- 2) Design: Create wireframes, mockups, and user experience workflows.
- 3) Development: Write the code for front-end and back-end.
- 4) Testing: Debug and ensure functionality across devices and browsers.
- 5) Deployment: Publish the website using hosting services.
- 6) Maintenance: Regular updates, performance monitoring, and adding new features.



Overview of web development

4. Modern Web Development Trends

- Progressive Web Apps (PWAs): Deliver an app-like experience in a browser.
- Responsive Design: Ensures websites look good on all devices.
- Serverless Architecture: Focuses on building applications without managing servers.
- AI and Chatbots: Incorporate intelligent systems for better user interactions.
- WebAssembly: Provides near-native performance for web applications.

Overview of web development

5. Popular Web Development Tools

a) Code Editors and IDEs

- VS Code, Atom, WebStorm.

b) Front-End Frameworks

- React, Bootstrap, Tailwind CSS.

c) Back-End Frameworks

- Express.js, Spring Boot, Ruby on Rails.

d) Version Control Tools

- Git, Bitbucket, GitHub.

d) Deployment Platforms

- Netlify, Vercel, Heroku, AWS.



Overview of web development

6. Skills Required for Web Developers

- Front-End Skills: HTML, CSS, JavaScript, responsive design.
- Back-End Skills: Server-side languages, database management, APIs.
- Problem-Solving: Debugging and fixing issues.
- Soft Skills: Collaboration, adaptability, and time management.

Overview of web development

7. Challenges in Web Development

- Cross-browser compatibility.
- Website speed and performance optimization.
- Cybersecurity threats.
- Keeping up with evolving technologies.

8. Career Opportunities

- Front-End Developer
- Back-End Developer
- Full-Stack Developer
- Web Designer
- UI/UX Designer

Introduction to the MEAN stack

- The MEAN stack is a collection of JavaScript-based technologies used to build dynamic, scalable, and robust web applications.
- The acronym stands for MongoDB, Express.js, Angular, and Node.js, which are the key components of the stack.
- Together, they provide a full-stack development environment, enabling developers to work seamlessly on both the front end and the back end using JavaScript.

Introduction to the MEAN stack

Key Components of the MEAN Stack

1. MongoDB (Database Layer)

- Type: NoSQL Database
- Role: Stores data in a flexible, JSON-like format called BSON (Binary JSON).
- Features:
 - Document-based storage (schema-less).
 - Scalable and efficient for large datasets.
 - Integrates well with JavaScript and the rest of the stack.



Introduction to the MEAN stack

- Use Case: Storing and retrieving data for applications like user information, product catalogs, or any other structured or semi-structured data.
- ## 2. Express.js (Back-End Framework)
- Type: Web application framework for Node.js.
 - Role: Handles the application's server-side logic and routes.
 - Features:
 - Lightweight and flexible.
 - Middleware support for request and response processing.
 - Simplifies handling HTTP requests and rendering responses.



Introduction to the MEAN stack

- Use Case: Building APIs and managing server-side functionalities.
- ## 3. Angular (Front-End Framework)
- Type: Client-side JavaScript framework developed by Google.
 - Role: Builds the user interface and manages the client-side logic of the application.
 - Features:
 - Component-based architecture.
 - Two-way data binding for seamless synchronization between the UI and application data.
 - Dependency injection and modular design.



Introduction to the MEAN stack

- Use Case: Developing interactive and dynamic single-page applications (SPAs)
- ## 4. Node.js (Runtime Environment)
- Type: JavaScript runtime environment built on Chrome's V8 engine.
 - Role: Executes JavaScript code on the server side.
 - Features:
 - Non-blocking, event-driven architecture.
 - Supports high concurrency and scalability.
 - Extensive package ecosystem through npm (Node Package Manager).
 - Use Case: Running the server, managing APIs, and connecting the front end to the database.

Introduction to the MEAN stack

Workflow in the MEAN Stack

1. Frontend (Angular):

- Angular handles the UI/UX and client-side logic.
- Sends HTTP requests to the server for data.

2. Server (Express.js & Node.js):

- Express.js processes client requests, applies middleware, and connects to the database.
- Node.js executes the server-side code.

Introduction to the MEAN stack

3. Database (MongoDB):

- MongoDB stores and retrieves data as requested by the server.
- The database uses JSON-like documents, making it easier to work with JavaScript throughout the stack.

4. Data Flow:

- Angular ↔ Express.js ↔ MongoDB.



Introduction to the MEAN stack

Advantages of the MEAN Stack

1. Single Language Across the Stack:

- Uses JavaScript for both client-side and server-side programming, simplifying development.

2. Open Source:

- All MEAN stack components are open-source, reducing development costs.

3. Scalability:

- Suitable for building high-performance, scalable applications.



Introduction to the MEAN stack

4. Ease of Development:

- MongoDB's JSON-like storage aligns naturally with JavaScript's data structures.
- Angular's two-way data binding simplifies UI development.

5. Fast and Efficient:

- Node.js's non-blocking I/O ensures fast request handling.
- MongoDB provides quick data storage and retrieval.



Introduction to the MEAN stack

Use Cases of the MEAN Stack

1. Single-Page Applications (SPAs):

- Examples: Dashboards, analytics tools.

2. E-commerce Platforms:

- Efficient for product catalogs and real-time order tracking.

3. Content Management Systems (CMS):

- Examples: Blogs, news sites.



Introduction to the MEAN stack

4. Social Media Platforms:

- Real-time updates and scalability for high user engagement.

5. Cloud-Native Applications:

- Scalable and performance-optimized applications hosted on the cloud.



Introduction to the MEAN stack

Tools and Libraries Commonly Used with MEAN

- **Mongoose:** ODM (Object Document Mapper) for MongoDB in Node.js.
- **RxJS:** Reactive programming library often used with Angular.
- **Postman:** API testing and development tool.
- **PM2:** Process manager for Node.js applications.
- **Jasmine/Karma:** Testing frameworks for Angular applications.

Introduction to the MEAN stack

Challenges of Using the MEAN Stack

1. Learning Curve:

- Developers need to understand multiple technologies (MongoDB, Express, Angular, Node.js).

2. Performance:

- For computationally heavy tasks, MEAN may not be the best choice due to JavaScript's single-threaded nature.

3. Data Migration:

- Moving structured data from relational databases to MongoDB can be challenging.



Setting up the development environment

1. Choose a Text Editor or Integrated Development Environment (IDE)

- Purpose: To write, edit, and debug code effectively.
- Popular Options:
 - VS Code: Lightweight, extensible, and widely used.
- Setup Steps:
 - Download from the official website.
 - Install necessary extensions/plugins (e.g., Prettier, ESLint for linting and formatting).



Setting up the development environment

2. Install a Web Browser

- Purpose: To test and debug your web applications.
- Popular Options:
 - Google Chrome: Includes Developer Tools for debugging.
 - Mozilla Firefox: Features advanced developer tools.
- Setup Steps:
 - Install the browser.
 - Familiarize yourself with Developer Tools (Ctrl+Shift+I for Chrome/Firefox).



Setting up the development environment

3. Install Node.js

- Purpose: Provides a runtime environment for JavaScript and includes npm (Node Package Manager) to manage dependencies.
- Setup Steps:
 - Download Node.js from its official site.
 - Install it using the provided installer.
- Verify installation:
 - `node -v`
 - `npm -v`

Setting up the development environment

4. Set Up Version Control

- Purpose: Tracks code changes and enables collaboration.
- Tool: Git
 - Setup Steps:
 - Download Git from its official site.
 - Install it with default configurations.
- Set up your Git username and email:
 - `git config --global user.name "Your Name"`
 - `git config --global user.email "youremail@example.com"`
 - Use platforms like GitHub, GitLab, or Bitbucket for hosting repositories.

Setting up the development environment

5. Install a Package Manager

- Purpose: Simplifies managing libraries and dependencies.
- Options:
 - npm: Comes with Node.js.
 - yarn: Alternative to npm, often faster.
- Setup Steps:
 - Yarn: Install using npm:
 - `npm install --global yarn`

Setting up the development environment

6. Set Up Front-End Tools

- Purpose: For styling, building UI components, and creating responsive designs.
- Tools:
 - CSS Frameworks: Bootstrap, Tailwind CSS.
 - Bundlers/Build Tools: Webpack, Vite, Parcel.
 - Preprocessors: Sass/SCSS for enhanced CSS.
- Setup Steps:
 - Install necessary frameworks or tools via npm:
 - `npm install bootstrap`
 - `npm install sass`

Setting up the development environment

7. Set Up a Back-End Framework

- Purpose: To build server-side applications.
- Popular Frameworks:
 - Express.js for Node.js.
- Install Express.js:
 - `npm install express`

Setting up the development environment

8. Install a Database

- Purpose: To store and manage data for your application.
- Options:
- MongoDB (NoSQL):
 - Download and install from the MongoDB website.
 - Use MongoDB Compass for a GUI-based database management tool.
- MySQL or PostgreSQL (SQL):
 - Install the database and a GUI tool like phpMyAdmin or pgAdmin.



Setting up the development environment

9. Set Up Development Servers

- Purpose: To test and run your application locally.
- Tools:
 - Live Server: For front-end projects.
 - Nodemon: Automatically restarts the server for back-end projects.
- Setup Steps:
 - Install Live Server extension in your IDE.
 - Install Nodemon globally:
 - `npm install -g nodemon`

Setting up the development environment

10. Install Additional Tools

- Task Runners: Automate repetitive tasks.
- Tools: Gulp, Grunt.
- Install Gulp:
 - `npm install --global gulp-cli`
- Linting Tools: Enforce coding standards.
- Tools: ESLint, Stylelint.
 - Install ESLint:
 - `npm install eslint --save-dev`



Setting up the development environment

11. Configure Environment Variables

- Purpose: Manage sensitive data (e.g., API keys, database credentials).
- Tools:
 - Use .env files with libraries like dotenv for Node.js:
 - `npm install dotenv`

12. Testing Frameworks

- Purpose: To ensure your application works as expected.
- Tools:
 - Jest: JavaScript testing.
 - Mocha: Node.js testing.
 - Cypress: End-to-end testing.



Setting up the development environment

- Setup Steps:
 - Install Jest:
 - `npm install jest --save-dev`

13. Set Up Deployment Tools

- Purpose: Host your application for public access.
- Options:
 - Netlify/Vercel: For front-end or full-stack apps.
 - Heroku/AWS: For full-stack or back-end apps.
- Use Git for deployment:
 - `git push heroku main`



Setting up the development environment

14. Optional Setup for Advanced Development

- Docker: Containerize your application for consistent deployment across environments.
- Install Docker from its official site.
- Virtual Machines: Simulate different environments.

15. Verify Your Setup

- Create a sample project:
 - `mkdir myproject`
 - `cd myproject`
 - `npm init -y`

× ○ DIGITAL LEARNING CONTENT



Parul[®] University



www.paruluniversity.ac.